Isolation Approaches for Parallel AI Coding Agents — A Deep Research Report

How apps and orchestrators let multiple agents work in the same codebase at once: mechanisms, implementations, and trade-offs. Compiled July 2026.

TL;DR

There is no single "isolation" feature — there are **four distinct problems** (file/git collisions, runtime interference, security containment, and merge coherence), and every tool on the market solves a different subset of them. **Git worktrees** have become the industry-default answer to file-level isolation: nearly every local orchestrator (Conductor, Superset, Claude Squad, Vibe Kanban, Cursor, Claude Code itself, Gas Town) provisions one worktree per agent, sharing the `.git` object store while giving each agent its own working directory, index, and branch. **Containers** (Sculptor, Dagger's Container Use, OpenHands, JetBrains Air) add environment and dependency isolation on top, at the cost of startup time and resource overhead. **Cloud sandboxes** (E2B, Modal, Daytona, Vercel, Northflank) push the boundary to the kernel or hypervisor level — gVisor userspace kernels or Firecracker microVMs — and are what the big async products (Codex cloud, Jules, Devin, Copilot coding agent) effectively are, wrapped in a PR-based workflow. **OS-level sandboxes** (Anthropic's `sandbox-runtime` using Seatbelt/bubblewrap, Codex CLI's Landlock/seccomp) answer a different question — *security* containment rather than parallel-work isolation — and increasingly ship as a complement to, not a replacement for, worktrees. The unsolved layer is **merge coherence**: worktrees prevent file collisions but not semantic conflicts, which is why the serious systems (Gas Town's Refinery, Multiclaude's CI ratchet, Clash's `git merge-tree` prediction) all add a merge-management layer on top. On the auth dimension, the January 2026 split matters: Anthropic blocked third-party harnesses from using Claude subscription OAuth, so "subscription orchestration" today means either wrapping the first-party CLI (Claude Code, Codex) or using API keys/OpenRouter — ChatGPT Plus, Copilot, and GitLab Duo subscriptions remain usable in third-party tools like OpenCode.

---

1. What "isolation" actually has to cover

Most discussions of parallel agents treat isolation as one thing. It is at least four. When two agents run against the same repository at the same time, the failure modes stack in layers: at the filesystem layer they overwrite each other's edits and corrupt the git index with concurrent `git add` operations; at the git layer an agent's mental model of the codebase diverges from what is actually on disk, producing confidently wrong reasoning (the Zylos research note calls this "context contamination" and "conversation confusion") (Zylos) . Below that sits the runtime layer, which is where the real-world pain concentrates: two agents sharing ports, environment variables, package caches, a development database, or a Stripe test account do not collide in git at all — they collide in the operating system, and they fail silently (Nimbalyst) . And below *that* sits the security layer: an agent that is prompt-injected, or simply careless, can delete your home directory or exfiltrate `~/.ssh` unless something constrains what the process can touch (Michael Livshits) .

The fourth layer is the one nobody's isolation primitive solves: **merge coherence**. Two agents editing the same function in separate worktrees will merge cleanly through git's text-level merge and produce a broken build — the isolation held perfectly and the system still failed (Nimbalyst) . Every serious orchestrator

therefore pairs an isolation primitive with a coordination mechanism (task claiming, dependency graphs, merge queues, CI gates), and the quality of *that* pairing is mostly what differentiates them.

Isolation dimension	The question it answers	Typical failure without it	Primitive that solves it
Filesystem / working tree	Can agent A' s edits clobber agent B' s?	Overwritten files, corrupted index, poisoned context	Git worktree, clone, container volume, VM disk
Runtime / services	Do agents share ports, DBs, env vars, caches?	Port 3000 conflicts, cross-contaminated test state, phantom Stripe charges (Nimbalyst)	Containers, per-worktree `.env`/port maps, DB branches, VMs
Security / blast radius	What can a rogue or injected agent reach?	Deleted home dir, exfiltrated keys, lateral movement (MediumMedium)	OS sandbox (seatbelt/ bubblewrap/Landlock), microVM, no-credentials-inside design
Merge / semantic	Does the combined result actually work?	Clean git merge, broken build; "merge wall" at 20+ agents (substack.com)	Task decomposition, merge queues, CI ratchets, conflict prediction

A useful way to read the entire market: **each product picks a point on the spectrum for dimension 1, bolts on ad-hoc answers for dimensions 2–3, and differentiates (or doesn't) on dimension 4.** The rest of this report walks the spectrum mechanism by mechanism, then covers the merge layer and the subscription-versus-API auth dimension, and ends with a consolidated tool matrix and selection guidance.

---

2. The spectrum at a glance

Isolation mechanisms form a rough continuum from "nothing but convention" to "a separate computer per agent," with isolation strength and setup cost rising together — except at the cloud end, where pre-warmed snapshots and copy-on-write tricks have collapsed startup times that used to be the decisive argument against heavy isolation (MediumMedium) .

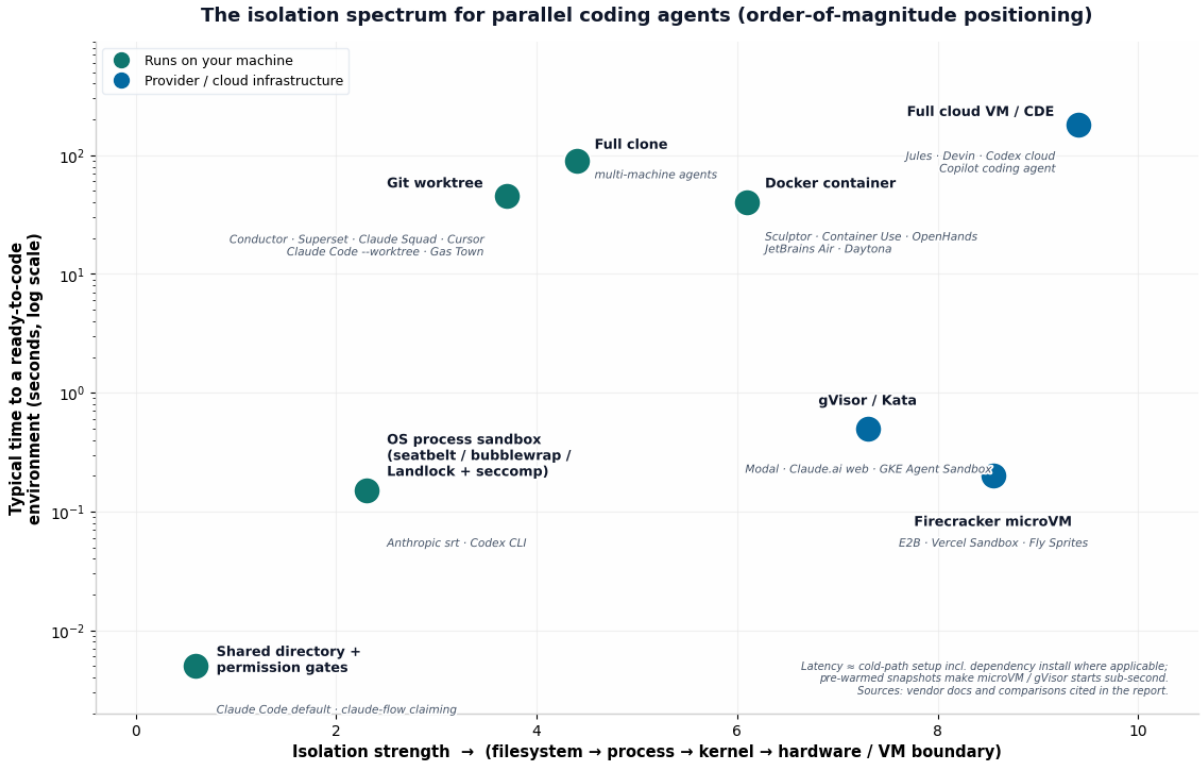

Figure 1: The isolation spectrum for parallel coding agents

Two things stand out from the positioning map. First, **git worktrees occupy an awkward middle**: they are heavier than doing nothing (each worktree needs dependency setup) yet they isolate strictly less than a container, because the agent still shares your OS user, your ports, your Docker daemon, your global package caches, and your kernel. Sculptor's co-founder summarized the motivation for skipping them entirely: worktrees meant "reinstalling dependencies with git worktrees" per agent, merge conflicts with multiple agents, and no protection against "Claude Code deleting your home directory" (Hacker News) . Second, the **cloud microVM tier inverts the old cost curve** — a pre-warmed Firecracker snapshot starts in ~150 ms, faster than `npm install` finishes printing its first line, which is why the newest local tools (JetBrains Air, Superset's planned cloud workspaces, claude-mpm's "cloud sled" roadmap) are all drifting toward remote isolation even for interactive work (Github) .

The taxonomy below names each mechanism, what it does and does not isolate, and who uses it; the sections after it go mechanism by mechanism.

Mechanism	Isolates	Does **not** isolate	Representative implementations
Shared directory + permission gates	Nothing structural; human approval is the gate	Files, git, runtime, secrets	Claude Code default mode; claude-flow' s in-memory task claiming (substack.com)
OS process sandbox	Process filesystem writes + network egress	Nothing about *which files* two agents edit	Anthropic `srt` (Seatbelt/ bubblewrap) (Github) ; Codex CLI (Landlock/ seccomp/restricted tokens) (Michael Livshits)

Table 2 – continued

Mechanism	Isolates	Does **not** isolate	Representative implementations
Git worktree	Working tree, index, HEAD per agent	Runtime, ports, DBs, untracked files, semantic conflicts	Conductor, Superset, Claude Squad, Vibe Kanban, Cursor, Claude Code, Gas Town (Nimbalyst)
Full clone	Everything a worktree does + independent remotes/config	Same-machine runtime; disk cost	Multi-machine setups (Agentrooms) (Github) ; naive fallback before worktree tooling
Container (Docker/ devcontainer)	Filesystem, deps, env vars, network namespace	Host kernel; Docker daemon if socket-mounted	Sculptor (rywalker.com) , Container Use (InfoQ) , OpenHands (Github) , JetBrains Air (Nimbalyst)
Userspace kernel / microVM sandbox	Kernel boundary; near-VM isolation	Orchestration/merge (out of scope —infra primitive)	E2B (Firecracker), Modal (gVisor), Daytona, Vercel, Northflank, Fly Sprites (GitHub Gist)
Full cloud VM / CDE per task	Entire machine; clean-slate environments	Merge coherence; repo must be pushed/PR' d back	Jules (Morph AI) , Devin, Codex cloud (wikipedia.org) , Copilot coding agent, claude-mpm cloud sled (Github)
Terminal multiplexer (control plane)	Nothing itself —it *operates* the above	—	Herdr (Github) , Claude Squad, amux (amux) , Gas Town' s tmux layer

3. Mechanism deep dives

3.1 Shared directory + permission gates (the baseline "no isolation" approach)

The default way most people run a coding agent is also the least isolated: the agent works directly in your checkout, and the safety mechanism is a **permission prompt** — every file write or bash command gets approved by a human, or by a permission mode like "accept edits." Nothing prevents two concurrent agents from editing the same file, and nothing prevents one agent from `rm -rf`-ing outside the project if the human rubber-stamps the 47th prompt of the hour, which is exactly the failure mode sandboxing guides warn about (Claude Code Camp) . Claude Code's Agent Teams notably defaults to this model for teammates: they share one working directory unless you opt into worktrees, and coordination happens through a file-based task list with owner-checked updates rather than through filesystem separation (alexlavaee.mealexlavaee.me) .

The more structured variant of "no isolation" is **task claiming**: claude-flow, for example, runs agents

against one shared codebase with no worktrees at all — an agent claims a task ("I'm working on this, don't touch it"), the claim is released if the agent dies, and correctness rests on the discipline of the claiming protocol (substack.com) . This is cheap and fast — zero setup, zero disk overhead, no merge step — and it is genuinely fragile under contention, because the claim is advisory rather than enforced; nothing at the filesystem level stops a confused agent from editing a claimed file. CC Mirror's unlocked Claude Code orchestration hardens the same idea with **ownership metadata on task JSON files** (only the owner or team-lead may update a task) and `blockedBy`/`blocks` dependency edges, which prevents coordination races even though the files themselves remain shared (theunwindai.com) .

Pros: zero setup, zero startup latency, no merge step, works in any repo instantly, minimal disk. **Cons:** no enforcement against cross-agent edits, index corruption risk, no security boundary, scales poorly with agent count — practitioner reports put reliable operation at 3–5 parallel agents, with 30–40% of runs hitting failure modes when pushing 10 agents on one repo in early multi-agent features (FindSkill.ai - AI Skills Directory) . This approach is fine for one agent plus a careful human; it is the wrong foundation for a fleet.

3.2 OS-level process sandboxes (seatbelt / bubblewrap / Landlock)

This layer answers a different question than the rest of the report — "what can this process touch?" rather than "how do agents avoid touching each other?" — but it matters here for two reasons: it is the standard complement to worktree isolation (isolate the work *and* cage the worker), and Anthropic open-sourced it as `@anthropic-ai/sandbox-runtime` (`srt`), so any orchestrator can adopt it (Github) .

Implementation. `srt` wraps any command with OS-native primitives: dynamically generated **Seatbelt profiles via `sandbox-exec` on macOS**, and **bubblewrap with network-namespace isolation on Linux**. The model is dual and secure-by-default: filesystem *writes* are allow-only (denied everywhere unless explicitly granted, e.g. the project directory and `/tmp`), filesystem *reads* use a deny-then-allow pattern, and **network access is allow-only**, routed through HTTP and SOCKS5 proxies on the host that enforce domain allowlists — on Linux the sandbox's network namespace is removed entirely so the only path out is a bind-mounted Unix socket to the proxy (Github) . OpenAI's Codex CLI implements the same philosophy with different primitives — Landlock + seccomp on Linux, Seatbelt on macOS, restricted tokens and ACLs on Windows — with network disabled by default and writes limited to the active workspace (Michael Livshits) . Measured overhead is roughly **80–160 ms of fixed per-invocation cost** (proxy spawn + policy load), negligible against any command that does real work (Claude Code Camp) . Anthropic's own products stack these by surface: Claude.ai runs in gVisor, Claude Code locally in Seatbelt/bubblewrap, and Claude Cowork in a full VM (Apple's Virtualization.framework on macOS, HCS on Windows) — with the design principle that "if credentials never enter the sandbox, they can't be exfiltrated" (Simon Willison's Weblog) .

Pros: near-zero startup cost; no Docker daemon or VM required; enforceable (kernel-level, not advisory); composable with any workspace strategy — you can sandbox an agent *inside* its worktree; open-source and reusable for MCP servers and arbitrary subprocesses (npm) . **Cons:** it does nothing for the parallel-work problem (two sandboxed agents still collide in a shared directory); policy authoring is fiddly; the failure mode is soft — Claude Code warns and runs unsandboxed if prerequisites are missing unless you set `sandbox.failIfUnavailable: true` (Claude Code Camp) ; and platform coverage is uneven (Linux needs `bubblewrap` + `socat` installed; Windows relies on WSL2 or, for Codex, newer restricted-token controls (wikipedia.org)). Think of it as the seatbelt, not the garage.

3.3 Git worktrees (the dominant pattern)

If one mechanism "won" 2025–2026, it is this one. A **git worktree** is a linked working directory that shares the main repository's `.git` object database: each worktree gets its own `HEAD`, index, and checked-out files, while commits, blobs, refs, remotes, hooks, and config stay centralized — internally, the new directory holds a `.git` *file* pointing back to `.git/worktrees/<name>`, with git splitting state between `$GIT_DIR` (per-worktree) and `$GIT_COMMON_DIR` (shared) (Zylos) . Versus a full clone, creation is fast and cheap — no duplicated history, no remote re-tracking — which is why it displaced "just clone twice" as the default (DEV Community) .

How the tools implement it. Claude Code ships first-class support: `claude --worktree <name>` branches from the default remote branch into `<repo>/.claude/worktrees/<name>` (branch `worktree-<name>`), auto-removes clean worktrees on exit, and prompts about dirty ones; subagents accept `isolation: "worktree"` in frontmatter, and a `WorktreeCreate` hook can run per-worktree setup commands (Zylos) . The background-agent lifecycle is: branch from current `HEAD` → check out into a temp worktree → run → report worktree path and branch back → auto-clean if no changes (Claude Code) . Cursor 3's Agents Window does the same via `/worktree` (plus `/apply-worktree`, `/delete-worktree`, and `/best-of-n` to race multiple models on one task, each in its own worktree), and also offers cloud sandboxes, remote SSH, or plain local directories as alternative session environments (byteiota.com) . OpenAI's Codex app exposes Local / Cloud / Worktree modes, with per-environment sandbox settings (Push To Prod) . The orchestrator class — Conductor (worktree per workspace, pass-through of your existing Claude login) (Run a team of coding agents on your Mac) , Superset (agent-agnostic: "if it runs in a terminal, it runs in Superset," with workspace setup/teardown scripts that can install deps, copy env vars, or even spin up a Neon/Supabase DB branch per workspace) (Hacker News) , Claude Squad (tmux + worktrees, AGPL) (Nimbalyst) , Vibe Kanban, Nimbalyst, Parallel Code (adds same-task "Arena mode"), Emdash, Gas Town, gnhf's `--worktree` flag — is essentially a management layer over this one primitive (Github) .

The pitfalls (this is where tutorials stop too early).

1. **Untracked files do not travel.** A fresh worktree contains tracked files only: no `node_modules`, no `.venv`, no `.env`, no local credentials (Run a team of coding agents on your Mac) . Fixes, ranked by popularity: per-workspace setup scripts (`npm ci --prefer-offline`, copy `.env` — this is what Superset's presets, Codex's `.codex/setup.sh`, and incident.io's wrapper function do) (AI Coding Tool for Developers) ; Conductor's "Files to copy" / `.worktreeinclude` globs for gitignored files (Run a team of coding agents on your Mac) ; **copy-on-write clones** on APFS (`cp -c`, near-zero cost for read-heavy trees) (Atomic Spin) ; or symlinking shared `node_modules` — which corrupts the shared folder the moment two worktrees install different versions, so only for identical dependency sets (AI Coding Tool for Developers) .
2. **One branch, one worktree.** Git refuses to check out the same branch twice; at scale you need unique branch names per session (`session/<sha>`) or detached `HEAD` — the workaround Codex uses (Tutorials Dojo) .
3. **Worktrees share the runtime.** Ports, databases, env vars, package caches, and test state are all still shared — five agents each running `npm run dev` on port 3000 fail loudly, five agents against one dev Stripe account fail silently (Nimbalyst) . Serious setups allocate port ranges, per-worktree `.env`, and scratch databases (or containerized/DB-branch-per-workspace services) before fanning out (Hacker News) .
4. **Worktrees defer conflicts, they don't resolve them.** File-level isolation is not logic-level isolation; the practical heuristic is "different files → parallel, same files → sequential or split first" (Tutorials Dojo) . Merge-side tooling is covered in §4.

Pros: fast (sub-second creation), disk-light (shared object store), native git semantics (every agent's work is a reviewable branch/PR — the same workflow as human teammates) (Tutorials Dojo) , zero infrastructure, works offline, tool support is now universal. **Cons:** no runtime/dependency isolation, setup-script tax per worktree, no security boundary (agent runs as your user with full filesystem reach — hence pairing with §3.2), cleanup burden (`git worktree prune`), and branch-name bookkeeping at scale. Reliability in practice tops out around a handful of agents before *human* review bandwidth, not git, becomes the binding constraint (FindSkill.ai - AI Skills Directory) .

3.4 Full clones and multi-machine isolation

The most brute-force approach — every agent gets its own `git clone` — is mostly superseded by worktrees on a single machine, because clones duplicate the object store, re-track remotes independently, and drift in config and hooks (DEV Community) . It survives in exactly the places worktrees cannot reach: **different machines**. Agentrooms/OpenAgents, for instance, is built around the idea that a frontend agent lives on your laptop, an infra agent on a Mac mini, an ML agent on a GPU box, and a reviewer on a build server — each with its own working directory *and* its own hardware, coordinated through a shared workspace protocol (Github) . This is isolation by physical separation: strongest possible runtime independence (nothing short of a shared network is common), and the only approach that also isolates **compute** — relevant if your fan-out is bounded by local RAM and CPU rather than by git.

Pros: total independence (hardware, OS state, daemons); scales past one machine's resources; each environment can be purpose-built (GPU box vs. build server). **Cons:** heaviest setup per agent; no shared object store (sync via push/pull, so "see each other's work" requires a network round-trip); you are now administering N machines, which is precisely the ops burden the cloud-VM products in §3.7 sell you out of. A pragmatic middle ground seen in the wild: one beefy always-on machine (a Mac mini or a small server) running all agents in worktrees, with thin clients (phone, laptop) attaching remotely — Herdr's `--remote` mode exists specifically to make that feel local (Github) .

3.5 Containers per agent (Docker / devcontainers)

Containers fix the two things worktrees cannot: **dependency/runtime isolation** (each agent gets its own installed environment, env vars, and network namespace) and **partial security isolation** (the agent's filesystem reach is limited to mounted volumes). The flagship implementation is Imbue's **Sculptor**: every agent runs in its own Docker container built from the repo's **devcontainer spec**, into which the repository is cloned; dependencies are baked into a cached image layer (install first, copy code second) so new agents start in seconds rather than minutes, and "Pairing Mode" bidirectionally syncs a container's state to your local IDE so you can test an agent's work without it ever touching your machine (imbue.com) . Sculptor also snapshots containers at turn boundaries for rollback, detects merge conflicts, and can hand resolution back to the agent (rywalker.com) . Dagger's **Container Use** takes the same idea library-first: an open-source MCP tool that gives any compatible agent its own containerized sandbox *plus* a git worktree, with `container-use list/watch/log/diff` for the human to inspect each environment (InfoQ) . **OpenHands** runs its agent loop against a pluggable "runtime" — a Docker image locally, or remote runtimes such as Modal — and its Agent Canvas fronts local, Docker, VM, and cloud backends interchangeably; running it *without* the sandbox is explicitly warned against, because the agent then has full host filesystem access (Github) . JetBrains Air offers Docker **or** worktree isolation per task in the same UI (Nimbalyst) , and Anthropic's own security guidance for running Claude Code unattended (`--dangerously-skip-permissions`) is: do it inside a devcontainer (Code with Andrea) .

The subtle failure mode of container-per-agent on one host is the **shared Docker daemon**. If each

agent's container talks to the host daemon — or worse, has the socket mounted — containers collide on names, networks, volumes, and port bindings, and an agent with daemon access can start a privileged container with host mounts and walk straight out of the sandbox (firecrawl.dev) . One engineer's multi-workspace odyssey through network namespaces, systemd-nspawn, ACLs, and SSH process management is a good catalogue of everything that leaks when you DIY this: identical compose project names, shared Postgres, ports bound host-wide, background processes killed with the session (Medium) . The production-grade answers are: one containerized Postgres with a separate logical DB per agent (Sculptor's suggestion) (Hacker News) , **DB branching** services (Neon/Supabase branch per workspace, as Superset's setup scripts do) (Hacker News) , or daemon-per-sandbox designs like Docker Sandboxes, which give each sandbox its own microVM-backed private Docker daemon and thereby solve docker-in-docker without privileged mode (firecrawl.dev) .

Pros: real dependency and runtime isolation; reproducible environments (every agent starts from the same image — "if one agent can run your tests, they all can") (imbue.com) ; meaningful security boundary when the socket is not mounted; snapshot/rollback semantics; image caching makes steady-state startup seconds-level. **Cons:** first-build cost in minutes; disk and RAM multiply per agent (each container holds a full repo copy + deps); orchestrating stateful services (databases, emulators) across containers is genuinely hard (Hacker News) ; iOS/simulator and GUI testing break out of the container model (Code with Andrea) ; weaker than VM isolation — a kernel escape is in the threat model, which is why untrusted multi-tenant workloads graduated to §3.6 (MediumMedium) .

3.6 Cloud sandbox infrastructure (gVisor, Kata, Firecracker microVMs, isolates)

Beneath the cloud agent *products* sits a now-crowded infrastructure tier selling "a computer for an agent" as an API: E2B, Modal Sandboxes, Daytona, Northflank, Vercel Sandbox, Fly.io Sprites/Machines, Blaxel, Runloop, CodeSandbox SDK, Cloudflare Sandboxes, plus CDE incumbents (Gitpod Flex, Coder) repositioning as agent runtimes (GitHub Gist) . Their distinguishing axis is the **isolation technology**, which maps directly onto the threat model: prompt-injection escape, resource abuse, and lateral movement between tenants (MediumMedium) .

Implementation spectrum. Docker containers isolate with namespaces and cgroups on a **shared host kernel** — fast (~90–500 ms cold), cheap (tens of MB per sandbox), but a kernel vulnerability is a cross-tenant escape (MediumMedium) . **gVisor** (Modal, Google Cloud Run, Claude.ai's web sandbox, GKE's Agent Sandbox) reimplements the Linux syscall surface in a userspace Go kernel: the container never talks to the real kernel, eliminating the shared-kernel escape at software level — lighter than a VM but with the caveat that a bug in gVisor itself is the new attack surface (Medium) . **Firecracker microVMs** (E2B, Vercel Sandbox, Fly, CodeSandbox) give each execution **its own kernel** via KVM with a ruthlessly minimal device model — ~100K lines of VMM code versus QEMU's ~2M, six virtio devices, <125 ms boot, ~5 MB memory overhead per VM — the same technology isolating AWS Lambda (MediumMedium) . The startup-latency war was won with **pre-warmed snapshot pools** (boot VMs ahead of time, snapshot memory, restore on request: E2B ~80–200 ms) (e2b.dev) and, at the research edge, **copy-on-write VM forks** — ZeroBoot `mmap(MAP_PRIVATE)`s a golden snapshot and forks KVM state, reaching **0.79 ms spawn with ~265 KB marginal memory per sandbox** (MediumMedium) . Cloudflare skips the OS entirely with V8 isolates (<50 ms, but JS/WASM only) (fast.io) .

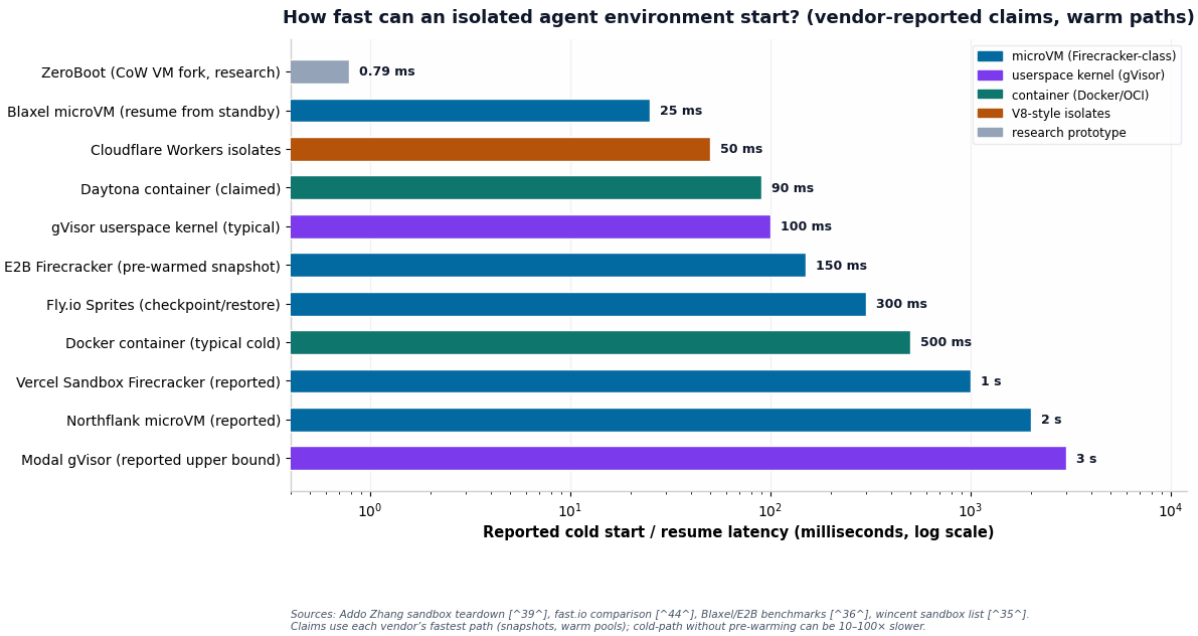

Figure 2: Reported cold-start latencies by sandbox technology

The operational details matter as much as the hypervisor. **Persistence models** diverge sharply: Daytona workspaces are stateful and long-lived by design; E2B is session-based with pause/resume/snapshot (pausing costs ~4 s per GiB of RAM, and sessions cap at 1 h on Hobby / 24 h on Pro, after which in-memory state is gone) (blaxel.ai) ; Fly Sprites persist 100 GB NVMe volumes with ~300 ms checkpoint/restore and scale-to-zero (GitHub Gist) . **Secrets handling** is its own sub-discipline — the recommended pattern is a credential proxy that injects scoped, short-lived tokens at request time (Cloudflare's Worker-side injection, Vercel Connect) rather than baking long-lived keys into sandbox images (developersdigest.tech) . And **GPU support** is sparse: Modal and Daytona offer it, E2B's managed tier does not, which decides the question for agents that run local models inside the sandbox (spheron.network) .

Pros: hardware- or userspace-kernel-enforced isolation suitable for *untrusted* code; sub-second starts from warm pools; clean API lifecycle (create/pause/snapshot/fork/destroy) that orchestrators can drive programmatically; per-second billing matches bursty agent fleets; BYOC/self-host options (E2B, Northflank, Daytona enterprise) for data-sovereignty constraints (developersdigest.tech) . **Cons:** you are building or buying orchestration on top — these are primitives, not products (no merge story, no review UX); costs scale with concurrency × duration and idle/standby billing surprises are a common production complaint (blaxel.ai) ; network egress policy, secrets injection, and cleanup are yours to design; and the vendor landscape is volatile (Daytona took its production codebase closed-source in June 2026) (Northflank) . This tier is what you reach for when agents run *other people's* code or prompts, or when you are productizing agent execution — less so for three agents on your own repo.

3.7 Full cloud VMs and CDEs per task (the async agent products)

The consumer-facing expression of the sandbox tier is the **async coding agent**: you assign a task, a fresh cloud machine is provisioned with your repo cloned into it, the agent works unattended, and the result returns as a pull request. **Jules** is the cleanest documented example — a task pool feeding a fleet of ephemeral Google Cloud VMs: scheduler provisions a VM, clones the repo, applies your saved **Environment Snapshot** (dependencies, install scripts, env vars) so setup is warm, plans with a strong model, executes and tests (the VM has network access for package installs), opens a PR, and **destroys**

the VM; 2026 updates close the loop by feeding CI failures back for autonomous fixes, with tiers up to 60 concurrent tasks (digitalapplied.com) . **OpenAI Codex cloud** runs each task in a separate cloud environment preloaded with the repository, internet access optional and off by default for security, with command logs and test results returned for review; its sandbox story is OS-level controls (including a Windows-native sandbox with restricted tokens and ACLs) plus the network policy (wikipedia.org) . **Devin** dedicates a cloud VM per session with shell, editor, and browser, and sells parallelism as "an army of Devins" for migrations and refactors (devin.ai) . GitHub's Copilot coding agent follows the same shape inside Actions-driven environments, and Cursor's background agents run on remote machines while its local sessions use worktrees — one product, two isolation tiers (Medium) .

The defining trade of this tier is **the PR as the isolation boundary**: isolation is total (clean VM, no shared state, no local resource consumption — "running a five-hour migration across ten services at once is viable here; it is a laptop fire with synchronous tooling" (digitalapplied.com)), but the feedback loop is asynchronous by construction. You cannot pair with the agent mid-task; you review a diff at the end. Sculptor's team frames this as the core UX split in the category — cloud-PR tools work well for tasks agents can one-shot, and feel clunky for complex work that needs turn-by-turn collaboration, which is why they built container isolation with live IDE sync instead (Hacker News) . The local-orchestrator world is converging toward the same infrastructure from the other direction: claude-mpm's roadmap has the local "PM" agent provisioning ephemeral cloud dev environments (Codespaces, Gitpod, Daytona, remote containers) per delegated task while coordination state persists locally — "ephemeral environments, persistent coordination" (Github) .

Pros: strongest practical isolation (dedicated machine, destroyed after use); zero local resource usage — your laptop can be off; clean-slate reproducibility via environment snapshots; natural audit trail (every task = one PR with logs); scales to tens of concurrent tasks without your hardware noticing (Morph AI) . **Cons:** async-only interaction (no mid-task pairing, though Jules lets you edit the plan before execution); requires pushing your code to a third party's infrastructure (private-code training exclusions and VM teardown claims notwithstanding (Morph AI)); environment setup is a new config surface to maintain; per-task pricing or subscription caps replace local compute as the constraint; and the merge step still exists — N concurrent tasks produce N PRs against a moving baseline, which is §4's problem all over again.

3.8 Terminal multiplexers: the control plane, not the isolation

Worth its own section because the category is easily confused with isolation itself. tmux-based tools — Claude Squad, amux, Herdr, Gas Town's session layer — primarily solve **process persistence and observability**: agents keep running when you detach, panes are organized into workspaces, and you can see which agent is working, blocked, or done. Herdr is the most explicit about the boundary: it is a single-binary multiplexer with persistent PTY sessions, mouse-native panes, agent-state detection (via process-name matching plus terminal-output heuristics, with deeper per-agent integrations), a JSON-over-Unix-socket API that lets **agents themselves** create panes, spawn helpers, and subscribe to state changes, and a `--remote` mode that makes a distant server feel local (preserving things plain ssh+tmux breaks, like pasting images into agents) (Github) . Its own marketing draws the line precisely: tmux "owns persistent terminal sessions but doesn't understand agents; desktop agent apps understand agents but live on one machine" (herdr) .

The key architectural point: **the multiplexer composes with any isolation primitive underneath.** Claude Squad pairs tmux panes with per-agent worktrees (Nimbalyst) ; Gas Town spawns a tmux session per worktree-hooked agent (substack.com) ; Herdr users report exactly the hybrid you'd expect — local

agents in one workspace, remote sandboxed agents attached over SSH in another, all visible in one sidebar (Hacker News) . Multiplexing without workspace isolation (tmux alone, or Kitty panes) gives you persistence and visibility while agents still share the directory — fine for agents on *different* repos, racy for agents on the same one. **Pros:** near-zero overhead, terminal-native, remote-friendly, scriptable (Herdr's socket API; tmux's CLI). **Cons:** zero filesystem/runtime/security isolation by itself; state detection is heuristic unless the agent integrates natively (Herdr's own support matrix shows "blocked" detection as partial for some agents) (Github) ; and the operator still does all merge management manually.

---

4. What isolation doesn't solve: the merge layer

Every mechanism above answers "how do agents avoid breaking each other *while working*?" None of them answer "how does the work come back together correctly?" This is where parallel-agent systems actually live or die, and it is worth treating as a first-class design layer rather than an afterthought.

Semantic conflicts survive perfect filesystem isolation. Two agents in two worktrees editing the same function produce branches that git merges cleanly into a broken build — text-level merge success, logic-level failure (Nimbalyst) . The defenses are organizational more than technical: decompose tasks so each agent owns a disjoint surface (different modules, different files); keep tasks on the same file sequential; and treat "one task depends on another's uncommitted output" as an automatic serial dependency, because worktrees cannot see each other's in-flight work (AI Coding Tool for Developers) . The codebase itself is a factor: modular boundaries and good test coverage make repos amenable to parallel-agent work for the same reasons they help human teams (Tutorials Dojo) . There is also early tooling for *predicting* conflicts before they happen — Clash runs `git merge-tree` across in-flight worktrees to pre-detect merge collisions (Zylos) .

The merge wall appears at scale. With 20 parallel workers producing 20 branches, every merge shifts the baseline and later PRs target code that has already changed (substack.com) . The systems built for this scale all institutionalize one of three merge strategies. Gas Town employs a dedicated **Refinery** — an agent that polls a review queue, merges what it can, and routes conflicts back to the original worker via the Beads ledger (issues as JSONL in git, hash-based IDs designed to avoid their own merge conflicts, cached in SQLite for queries) (mikemason.ca) . Multiclaude runs a **CI ratchet** (a "Brownian ratchet" philosophy): if CI passes, the PR merges, full stop; the daemon watches PR status and assigns fixes for failing PRs back to their authors — its singleplayer mode merges without human review at all (shipyard.build) . Claude Code's own ecosystem leans on **reviewer agents** as a quality gate — Addy Osmani's recommended ratio is roughly one read-only reviewer per 3–4 builders, auto-triggered on every task completion, so the human lead only ever sees green-reviewed code (addyosmani.com) . And practically everyone converges on making the PR the unit of accountability, because it reuses the review muscle teams already have (Tutorials Dojo) .

The sobering operational data point is that the **human becomes the bottleneck well before the tooling does**: reliable day-to-day operation today is 3–5 parallel agents; early adopters pushing 10 concurrent agents on one repo saw failure modes in 30–40% of attempts in the first weeks (FindSkill.ai - AI Skills Directory) , and every background agent completing is a review item landing in your queue (Claude Code) . Steve Yegge's Gas Town — 20–30 agents coordinated by a "Mayor" over Beads-tracked work — is simultaneously the existence proof that the scale is reachable and the cautionary tale: it is self-described "extremely alpha," has autonomously merged PRs despite failing integration tests, and once erased a production database's passwords (mikemason.ca) . Isolation mechanisms decide how *safe* parallelism is; the merge layer decides how *correct* it is; and review bandwidth decides how *much* of it

you can actually absorb.

---

5. The auth dimension: subscriptions vs. API keys vs. gateways

Because the question explicitly spans "directly but also through subscription," the isolation story has an economic shadow: *which credentials* power each agent determines what parallelism costs and even what is permitted.

The 2026 fault line. On January 9, 2026, Anthropic blocked third-party harnesses from spending Claude Pro/Max subscription quota: OpenCode's bundled plugin was removed (its docs now state plainly that "Anthropic explicitly prohibits this"), and tools like Pi began surfacing that API usage bills separately from the subscription — Claude-first developers on third-party harnesses are, in one reviewer's phrase, "forced to pay twice" (Thomas Wiegold - AI Solutions & Web Development) . The carve-out is telling: routing through **ACP** (Agent Client Protocol, e.g. Zed's integration) counts as "still using Claude Code," so the subscription works there, while reimplemented harnesses that call the API directly do not qualify (Hacker News) . Meanwhile the other vendors went the opposite way — OpenCode supports **ChatGPT Plus, GitHub Copilot, and GitLab Duo subscriptions with zero setup**, and its docs point that out while noting the Anthropic prohibition (opencode.ai) . OpenAI's own Codex app/CLI treats any ChatGPT Plus/Pro/Business/Enterprise/Edu login as valid credentials (OpenAI) .

What this means for orchestration. Wrappers that drive the *first-party CLI* inherit whatever that CLI is logged in with — Conductor's FAQ is explicit: API key if you use an API key, Pro/Max subscription if that is how your Claude Code is authenticated (Run a team of coding agents on your Mac) . So subscription-based parallelism is alive and well *inside* the official harnesses (Claude Code, Codex CLI), which is one more reason the orchestrator class (Superset, Claude Squad, Herdr, Gas Town) shells out to the official CLIs rather than reimplementing them. API-key auth remains the unrestricted path — every harness, every provider, including OpenRouter aggregation — but at metered prices, and the economics are stark: heavy Claude Code usage that costs $100–200/month on Max can exceed $1,000/month at API rates, which is precisely why the subscription moat exists (Hacker News) . A last wrinkle for fleets: subscription rate limits are per-account, so N parallel agents on one subscription share one token budget — a gateway like **agentgateway** sits between your tools and the providers to centralize auth (subscription passthrough *or* key routing), add per-agent rate limits, and give one observability point for the whole fleet (maniak.io) .

Auth route	Where it works	Parallelism implications
Claude Pro/Max subscription	Claude Code CLI and ACP-connected surfaces only; blocked in OpenCode/Pi-style harnesses since Jan 2026 (opencode.ai)	N agents share one rate-limit budget; official-CLI wrappers inherit it (Run a team of coding agents on your Mac)
ChatGPT Plus/Pro subscription	Codex CLI/app/cloud; also usable in OpenCode (OpenAI)	Same shared-budget caveat; broad third-party tolerance
GitHub Copilot / GitLab Duo subscriptions	First-party tools + OpenCode (opencode.ai)	Cheapest multi-agent fuel if already subscribed
Provider API keys (direct or OpenRouter)	Every harness, no client restrictions	Pay-as-you-go; costs scale linearly with agents; no subscription caps

Table 3 – continued

Auth route	Where it works	Parallelism implications
Local/open-weight models (Ollama etc.)	Aider, OpenCode, most harnesses	Marginal cost electricity; quality gap is the price; GPU decides feasibility (sanj.dev)

6. Consolidated tool matrix

The table aggregates every tool discussed, grouped by the isolation primitive it bets on. "Auth model" reflects §5's constraints as of July 2026; this category churns fast, so verify before committing.

Tool	Isolation primitive	Runs where	Agents supported	Auth model	Notes
Claude Code (native)	Worktree (`--worktree`, `isolation: worktree`); srt OS sandbox; Agent Teams shared-dir by default (Zylos)	Local	Claude only	Max/Pro subscription or Anthropic API	File-based team coordination; `WorktreeCreate` hook (alexlavaee.mealex
Codex app / CLI	OS sandbox (Landlock/ seccomp/ Seatbelt); worktree mode; cloud per-task containers (Push To Prod)	Local + OpenAI cloud	GPT-x only	ChatGPT subscription or OpenAI API	Per-environment permission profiles (OpenAI)
Cursor 3	Worktree (`/ worktree`), cloud sandbox, remote SSH, or local dir per session (byteiota.com)	Local + Cursor cloud	Multi-model	Cursor subscription	`/best-of-n` races models in parallel worktrees
Windsurf (Wave 13+)	Worktrees for parallel Cascade sessions (Nimbalyst)	Local	Multi-model	Subscription	Multi-pane monitoring

Table 4 – continued

Tool	Isolation primitive	Runs where	Agents supported	Auth model	Notes
Conductor	Worktree per workspace; "Files to copy" for gitignored env files (Run a team of coding agents on your Mac)	Local (macOS)	Claude Code, Codex, Cursor	Pass-through of existing CLI login (API or Pro/Max) (Run a team of coding agents on your Mac)	Cleanest worktree UX on Mac
Superset	Worktree per task + setup/ teardown scripts (deps, env, DB branches) (Hacker News)	Local (macOS; Linux beta)	Any CLI agent (Pi, Claude Code, Codex, Amp, Droid···)	Whatever each CLI uses	Source-available (ELv2); cloud workspaces planned (Github)
Claude Squad	tmux panes + worktree per agent (Nimbalyst)	Local (any tmux env)	Claude Code, Codex, Gemini, OpenCode, Amp, Aider	Whatever each CLI uses	AGPL; terminal-native
Vibe Kanban	Worktree-backed workspaces, kanban review flow (Nimbalyst)	Local	Broad (Claude Code, Codex, Gemini, Amp, Copilot, Cursor, OpenCode)	Whatever each CLI uses	Apache-2.0; company shut down Apr 2026, community-maintained (Nimbalyst)
Nimbalyst / Parallel Code / Emdash	Worktree per agent	Local desktop	Broad	BYO keys/logins	Parallel Code adds Arena mode (same-task races), MIT (Parallel Code)
Herdr	None itself (multiplexer); composes with worktrees/ remote sandboxes	Local + remote via `--remote`	Broad incl. Pi (native integrations), Claude Code, Codex, Kimi	Whatever each CLI uses	Agent-state detection + socket API agents can drive (Github)
amux	tmux + worktrees; kanban with atomic task claiming (amux)	Local server + web/PWA	Claude Code primary	CLI logins	Single-file Python server

Table 4 – continued

Tool	Isolation primitive	Runs where	Agents supported	Auth model	Notes
Gas Town	Worktree "hooks" + tmux; Beads ledger for state	Local	Claude Code, Copilot, Codex, Gemini, others	CLI logins	20–30 agent scale; Refinery merge agent; very alpha (mikemason.ca)
Multiclaude	Worktrees + CI-ratchet merging	Local	Claude Code	CLI login	Singleplayer auto-merge mode (shipyard.build)
gnhf	Optional `--worktree` per run	Local	Claude, Codex, Rovo Dev, OpenCode, Copilot, Pi, ACP	CLI logins	Overnight "ralph loop" iteration (Github)
Sculptor	**Docker container per agent** (devcontainer-cached images); snapshots; Pairing Mode sync (imbue.com)	Local (macOS/ Linux)	Claude Code (Codex added)	Claude Pro/Max or API key	Solves dep/ runtime isolation worktrees miss (rywalker.com)
Container Use (Dagger)	Container + worktree per agent, MCP-driven (InfoQ)	Local	Any MCP-compatible agent	Agent' s own	Library/CLI rather than app
OpenHands	Docker runtime (or remote runtimes e.g. Modal); VM/cloud backends (Github)	Local + self-host cloud	OpenHands agent + any ACP agent (Claude Code, Codex, Gemini)	Any LLM key	Warns against unsandboxed local runs (Github)
JetBrains Air	Docker **or** worktree per task (Nimbalyst)	Local (macOS preview)	Codex, Claude Code, Gemini CLI, Junie	CLI logins	IDE-vendor entrant
E2B / Vercel Sandbox / Fly Sprites	Firecracker microVM per session (GitHub Gist)	Cloud (SDK)	Bring your own agent	API keys	Infra primitive; pause/snapshot lifecycle (developersdigest.tech)
Modal Sandboxes	gVisor per session	Cloud (SDK)	BYO agent	API keys	GPU option (spheron.network)

Table 4 – continued

Tool	Isolation primitive	Runs where	Agents supported	Auth model	Notes
Daytona / Northflank	Containers (Daytona) / Kata·gVisor· Firecracker choice (Northflank) (Northflank)	Cloud / BYOC	BYO agent	API keys	Daytona closed-sourced Jun 2026 (Northflank)
Jules	Ephemeral Google Cloud VM per task, env snapshots, destroyed after (digitalapplied.com)	Google cloud	Gemini only	Google AI sub (free 15 tasks/ day → Ultra 60 concurrent) (Morph AI)	PR-only output; CI-failure auto-fix loop
Devin	Dedicated cloud VM per session (devin.ai)	Cognition cloud	Proprietary	$500/mo class	"Army of Devins" parallelism
Copilot coding agent	Actions-driven cloud environment per task (amux)	GitHub cloud	OpenAI/ Anthropic/ Gemini via Copilot	Copilot subscription	Native PR + Actions integration
Anthropic Managed Agents	Hosted environments; coordinator/ multiagent API (Claude Docs)	Anthropic cloud	Claude	API	Vault-scoped credentials per session
agentgateway	(Auth/ observability layer, not isolation)	Local/cluster proxy	Any OpenAI/ Anthropic-compatible tool	Subscription passthrough or key routing (maniak.io)	Per-agent rate limits, fleet observability

7. Choosing an approach

The honest answer is that most working setups **stack two or three mechanisms**, because each covers a different failure domain. The stack that has emerged as the 2026 default for a solo developer or small team is: **worktrees for file isolation + an OS sandbox for blast radius + setup scripts for environment parity + CI (or a reviewer agent) as the merge gate** — exactly the composition Claude Code now ships natively (`--worktree` + `/sandbox`) and that desktop orchestrators wrap in a GUI (Zylos) . Containers enter the stack when dependency isolation matters (agents must install different toolchains, or you want `--dangerously-skip-permissions`-level autonomy with a hard boundary) (Code with Andrea) , and cloud VMs/sandboxes enter when the work is untrusted, when local hardware is the constraint, or when the whole point is to close the laptop.

Scenario-by-scenario, the research points to fairly clean fits:

Your situation	Recommended stack	Why
2–4 agents, same repo, you' re at the keyboard	Worktree orchestrator (Conductor / Superset / Claude Squad) + `srt` sandbox	Fast, free, reviewable; ports/env handled by setup scripts (Hacker News)
Agents install conflicting deps or need `--dangerously-skip-permissions`	Container per agent (Sculptor / Container Use / devcontainer)	Dep isolation + a real security boundary (imbue.com)
Overnight unattended batches ("drain the backlog")	Async cloud agents (Jules/Codex cloud) or Gas Town-class worktree fleets behind CI gates	Zero local resources; PR-per-task audit trail; ratchet merging (digitalapplied.com)
Multi-hour, turn-by-turn collaboration with several agents	Local containers with IDE sync (Sculptor Pairing) or worktrees + multiplexer (Herdr)	Cloud-PR loop is too coarse for interactive work (Hacker News)
Running untrusted code/ prompts, or productizing agents	microVM/gVisor sandboxes (E2B/ Modal/Northflank), credential proxy, no long-lived secrets inside (developersdigest.tech)	Kernel-level boundary; sub-second warm starts
One always-on machine, phone/thin-client control	Headless box: worktrees + Herdr `--remote` (or amux PWA); cloud for overflow	Multiplexer gives persistence; worktrees give isolation (Github)
Constrained local hardware (e.g., 32 GB RAM desktop, iGPU)	Worktrees locally (cheap), push heavy parallel batches to cloud VMs	Disk per worktree repo + one dep install; containers/VMs multiply RAM (rywalker.com)
Subscription-maximizing (Claude Max / ChatGPT Plus)	Orchestrators that wrap the official CLIs (Conductor, Superset, Claude Squad, Herdr)	First-party CLI keeps subscription auth; third-party harnesses lose Claude' s (opencode.ai)
Cost-minimizing / provider-flexible	API keys or OpenRouter + OpenCode/Aider-class harnesses; local models for cheap loops	No client restrictions; linear cost; gateway for rate limits (maniak.io)

Two cross-cutting rules fall out of the research regardless of scenario. **First, decompose before you fan out** — the reliability ceiling (3–5 agents smooth, ~30–40% failure at 10) is dominated by task overlap and review bandwidth, not by any isolation mechanism (FindSkill.ai - AI Skills Directory) . **Second, treat runtime isolation as part of workspace provisioning, not as an afterthought**: port ranges, per-workspace `.env`, scratch or branched databases, and "no shared Docker socket" are the difference between a fleet that works and five agents silently corrupting one dev database (Nimbalyst) .

---

8. Where this is heading

Three convergences are visible as of mid-2026. **Local and cloud isolation are merging**: tools born local are adding remote backends (Superset's cloud workspaces, claude-mpm's cloud sled, Herdr's remote attach), while tools born cloud are adding interactive sync (Sculptor's Pairing Mode, Cursor's one-UI-over-four-environment-types) — the end state is one control plane over a continuum of isolation strengths

chosen per task (Github) . **Isolation is becoming default-on and boring**: first-party CLIs now ship worktrees *and* OS sandboxes built in (Claude Code, Codex), and the sandbox primitives themselves are open-source commodities (`srt`, Firecracker, gVisor) rather than differentiators (Github) . And **the competitive layer is moving up-stack to coordination and merge**: Beads-style git-backed state, refinery/ratchet merge agents, conflict prediction, and reviewer gates are where the differentiation — and the remaining alpha-stage danger — now lives (mikemason.ca) . The isolation question "how do agents not break each other?" is close to solved at every price point; the question being worked on now is "how does thirty agents' worth of work become one coherent, reviewed, mergeable change?" — and that is a coordination problem wearing an isolation costume.

Prepared July 17, 2026. This report is for general informational purposes; the agent-orchestration category is volatile — verify current features, pricing, and terms (especially subscription-usage policies) before adopting any tool.